

G02 Gate B - Clinic Supply Deliveries + Quality Cases

PROTECTED INDEPENDENT ASSESSMENT | SEALED - OPEN ONLY IF ASSIGNED

MySQL setup: G02_GATE_B_MYSQL_SOURCE_CANONICAL.sql

PostgreSQL setup: G02_GATE_B_POSTGRESOL_SOURCE_CANONICAL.sql

Before starting, create a new attempt ID in the Evidence & History Record. During the Gate, keep Teaching Books, Reviews, other Gate variants and AI solving closed. Your SQL can be imperfect; your evidence must show what you actually attempted.

Part 1 - Foundation proof (K1-K2)

1. State one-row grain and primary key for **clinics**, **deliveries**, **quality_events** and **delivery_exceptions**. Then use SELECT/aliases/ORDER BY/LIMIT to inspect each without changing source data.
2. Normalize **supply_raw** only in a derived result using TRIM + a consistent case rule. Count distinct normalized values. Count NULL **coordinator** separately and show a COALESCE display label without overwriting NULL.
3. Return only **Received** rows dated 2026-08-11 through 2026-08-15 inclusive. Use explicit parentheses if combining AND/OR. Prove the eligible row count independently.
4. Derive **net_value = units * unit_cost * (1 - rebate_rate)**. Add a CASE value band of your own documented thresholds. Explain the arithmetic/result grain and validate at least two rows manually.
5. GROUP BY clinic_id; find entities with at least 2 Received rows using HAVING. Then solve a different question with EXISTS or a subquery: which entities have at least one exception record?
6. Build one combined exception/activity stream using **UNION ALL** from delivery_exceptions plus event rows whose type represents an exception. State why UNION ALL or UNION is the correct choice for your evidence grain.

Part 2 - Integrated MySQL build (K3-K5)

Business request: produce exactly one row per **Zone** with: Received record count, Received net_value, exception-record count, event count, and the top Received delivery_id by net_value. If top values tie, use delivery_id ascending as the deterministic tie-break.

7. Before building it, demonstrate the raw deliveries LEFT JOIN quality_events fan-out using row counts. Explain why summing fact-level money after that raw join is unsafe.
8. Build grain-safe CTE/subquery stages. Put a SQL comment above each stage stating its row grain. Do not use DISTINCT as a patch for multiplied monetary totals.
9. Use a window function for top-per-group and make its ORDER BY deterministic. Also use LAG or LEAD over quality_events PARTITIONED by delivery_id and ordered by timestamp + event ID to show one event sequence safely. Validate the top row independently for each group and manually verify one event timeline.
10. Save independent controls for source rows, eligible rows, total eligible net_value, event/exception totals, final group row count and one key uniqueness check. Reconciliation must use a logically independent query path.

Part 3 - PostgreSQL database-thinking proof (K6)

11. Run the PostgreSQL setup. State schema/table grain and keys. Record the baseline row count and the count/sum where state='Posted'.
12. Inside BEGIN, change one state or amount; inspect it; ROLLBACK; prove the baseline returns. Deliberately attempt one invalid value that a CHECK/PK/FK constraint should reject; do not weaken the constraint.
13. Choose one repeated filter/join column. Justify indexing or not indexing it. If you create an index, save EXPLAIN before/after and state only what the observed plan/cost/row estimates show. "Index exists" is not proof that the query became faster.
14. Create a normal PostgreSQL view at one stable grain. Validate its row count/total with an independent query and explain why a normal view stores reusable query logic rather than an automatically cached independent copy.

Part 4 - timed defense (K7)

15. Close SQL reference material. In 10-15 minutes, explain: source grain, biggest fan-out risk, one Boolean/NULL trap, why your window order is deterministic, how your monetary reconciliation is independent, what ROLLBACK proved, what EXPLAIN did/did not prove, and what the view represents.

16. Freeze the attempt: save SQL files, outputs/control values, error/recovery note, time used, confidence and your own-words explanation. Only after this saved evidence exists may the matching Review be opened.

Gate decision

Do not calculate a pass percentage. Reviewer marks K1-K7 as DEMONSTRATED or REPAIR_REQUIRED, cites the saved evidence, records any critical defect, and follows the protected routing in Start Here.

Competency	Status	Evidence reference	Reviewer note / critical defect
K1 - Foundation SQL semantics	DEMONSTRATED / REPAIR_REQUIRED		
K2 - Transformation + grouped reasoning	DEMONSTRATED / REPAIR_REQUIRED		
K3 - Joins, cardinality + set logic	DEMONSTRATED / REPAIR_REQUIRED		
K4 - CTEs + window/ranking	DEMONSTRATED / REPAIR_REQUIRED		
K5 - Multi-grain reconciliation	DEMONSTRATED / REPAIR_REQUIRED		
K6 - PostgreSQL database thinking	DEMONSTRATED / REPAIR_REQUIRED		
K7 - Evidence + explanation discipline	DEMONSTRATED / REPAIR_REQUIRED		

Reviewer route after this attempt

If K1-K7 are demonstrated and no critical defect remains: GATE_FAMILY_PASS. Otherwise preserve this attempt, open only the matching Review, and assign targeted changed repair. The next full Gate remains sealed unless explicitly assigned.